COMPUTER ENGINEERING

QUEZON CITY

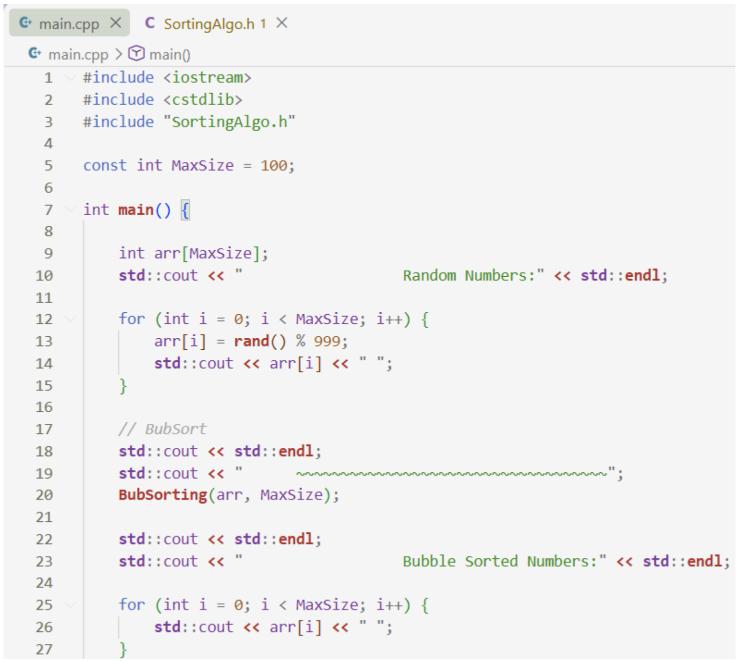
COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	September 4, 2026	**DATE SUBMITTED:**	September 4, 2026
NAME:	DE VERA, Sophia D.	**INSTRUCTOR:**	Engr. QUEJADO, Jimlord

HANDS-ON ACTIVITY 7.1 SORTING ALGORITHM PART 1

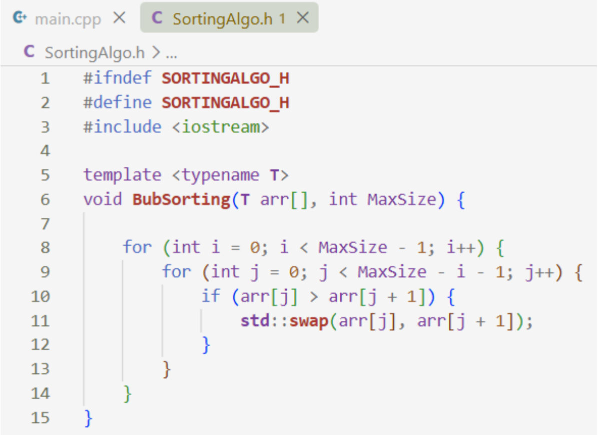
1. PROCEDURE OUTPUT

BUBBLE SORTING

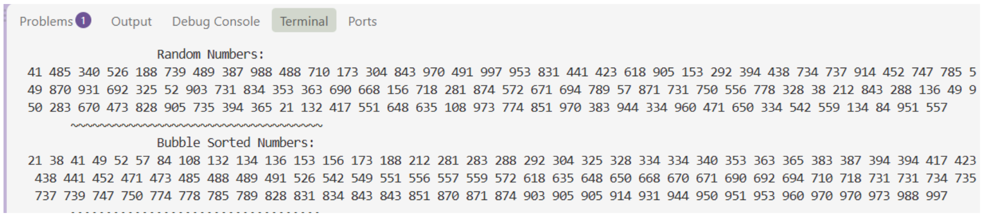
SCREENSHOTS



```cpp
#include <iostream>
#include <cstdlib>
#include "SortingAlgo.h"

const int MaxSize = 100;

int main() {

    int arr[MaxSize];
    std::cout << "                  Random Numbers:" << std::endl;

    for (int i = 0; i < MaxSize; i++) {
        arr[i] = rand() % 999;
        std::cout << arr[i] << " ";
    }

    // BubSort
    std::cout << std::endl;
    std::cout << "      ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~";
    BubSorting(arr, MaxSize);

    std::cout << std::endl;
    std::cout << "                  Bubble Sorted Numbers:" << std::endl;

    for (int i = 0; i < MaxSize; i++) {
        std::cout << arr[i] << " ";
    }
```



```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H
#include <iostream>

template <typename T>
void BubSorting(T arr[], int MaxSize) {

    for (int i = 0; i < MaxSize - 1; i++) {
        for (int j = 0; j < MaxSize - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

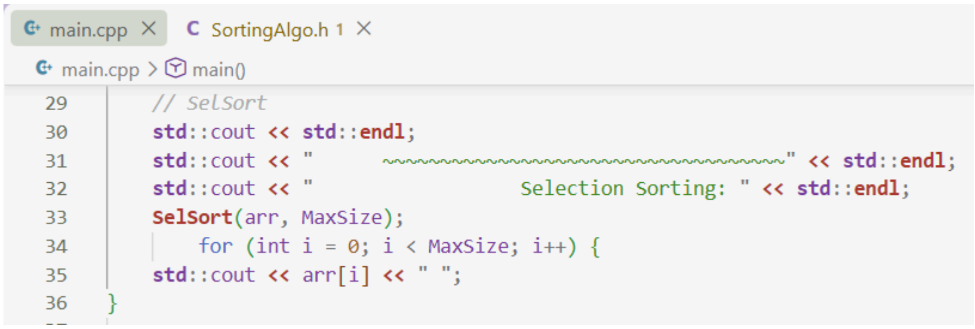


```
                  Random Numbers:
41 485 340 526 188 739 489 387 988 488 710 173 304 843 970 491 997 953 831 441 423 618 905 153 292 394 438 734 737 914 452 747 785 5
49 870 931 692 325 52 903 731 834 353 363 690 668 156 718 281 874 572 671 694 789 57 871 731 750 556 778 328 38 212 843 288 136 49 9
50 283 670 473 828 905 735 394 365 21 132 417 551 648 635 108 973 774 851 970 383 944 334 960 471 650 334 542 559 134 84 951 557
      ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                  Bubble Sorted Numbers:
21 38 41 49 52 57 84 108 132 134 136 153 156 173 188 212 281 283 288 292 304 325 328 334 334 340 353 363 365 383 387 394 394 417 423
 438 441 452 471 473 485 488 489 491 526 542 549 551 556 557 559 572 618 635 648 650 668 670 671 690 692 694 710 718 731 731 734 735
 737 739 747 750 774 778 785 789 828 831 834 843 843 851 870 871 874 903 905 905 914 931 944 950 951 953 960 970 970 973 988 997
```

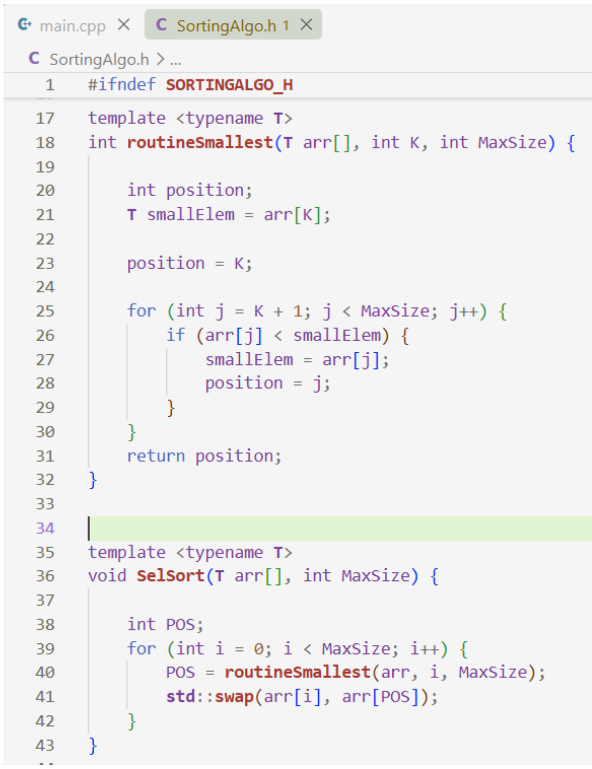
OBSERVATION ANALYSIS	- In this sorting technique, the bubble sort compares the two adjacent elements. It then continues to process the list repeatedly until the elements are arranged in the correct order. In my opinion, this has the longest programming runtime process since it sorts from start to finish.

SELECTION SORTING

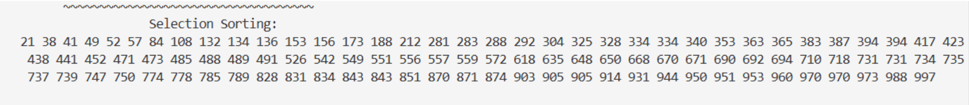
SCREENSHOTS



```cpp
    // SelSort
    std::cout << std::endl;
    std::cout << "      ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~" << std::endl;
    std::cout << "                  Selection Sorting: " << std::endl;
    SelSort(arr, MaxSize);
        for (int i = 0; i < MaxSize; i++) {
    std::cout << arr[i] << " ";
}
```



```cpp
#ifndef SORTINGALGO_H
template <typename T>
int routineSmallest(T arr[], int K, int MaxSize) {

    int position;
    T smallElem = arr[K];

    position = K;

    for (int j = K + 1; j < MaxSize; j++) {
        if (arr[j] < smallElem) {
            smallElem = arr[j];
            position = j;
        }
    }
    return position;
}


template <typename T>
void SelSort(T arr[], int MaxSize) {

    int POS;
    for (int i = 0; i < MaxSize; i++) {
        POS = routineSmallest(arr, i, MaxSize);
        std::swap(arr[i], arr[POS]);
    }
}
```



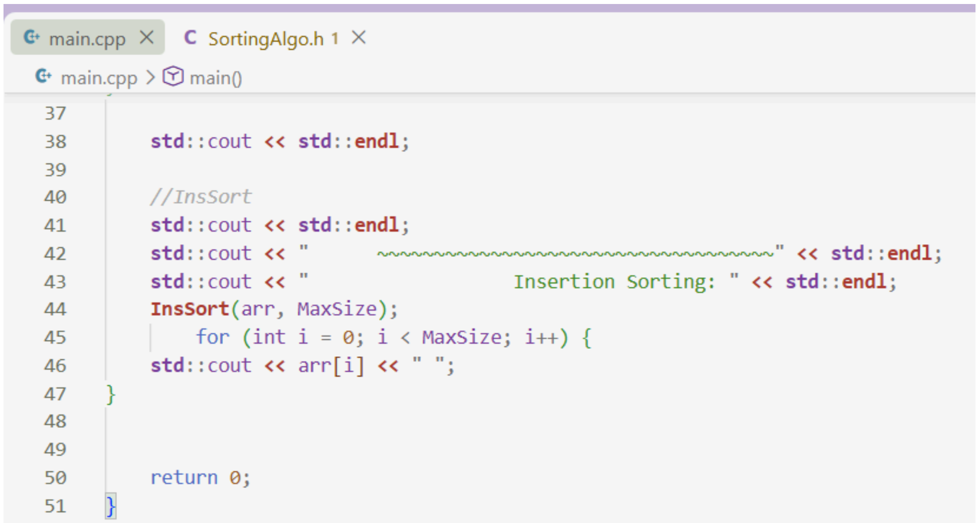
```
     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                    Selection Sorting:
21 38 41 49 52 57 84 108 132 134 136 153 156 173 188 212 281 283 288 292 304 325 328 334 334 340 353 363 365 383 387 394 394 417 423
 438 441 452 471 473 485 488 489 491 526 542 549 551 556 557 559 572 618 635 648 650 668 670 671 690 692 694 710 718 731 731 734 735
 737 739 747 750 774 778 785 789 828 831 834 843 843 851 870 871 874 903 905 905 914 931 944 950 951 953 960 970 970 973 988 997
```

OBSERVATION ANALYSIS

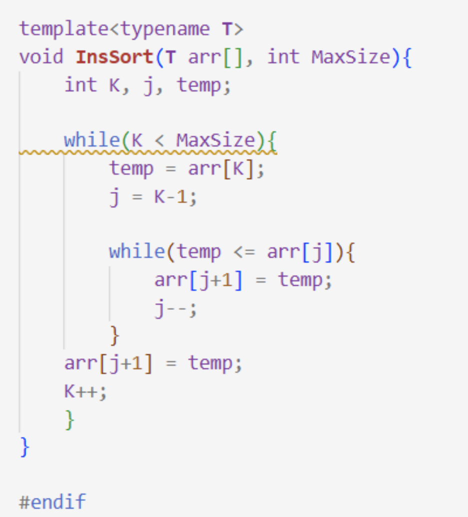
- The selection sorting on the other hand works by repeatedly finding the smallest element from the unsorted part of the list and placing it at the beginning of that part. I do find this relatively efficient compared to the former sorting method. Because once it's done on the smallest value element, it doesn't start back from the start of the array list.

INSERT SELECTION

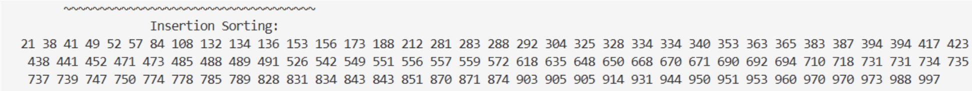
SCREENSHOTS



```cpp

    std::cout << std::endl;

    //InsSort
    std::cout << std::endl;
    std::cout << "          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~" << std::endl;
    std::cout << "                          Insertion Sorting: " << std::endl;
    InsSort(arr, MaxSize);
        for (int i = 0; i < MaxSize; i++) {
    std::cout << arr[i] << " ";
}


    return 0;
}
```



```cpp
template<typename T>
void InsSort(T arr[], int MaxSize){
    int K, j, temp;

    while(K < MaxSize){
        temp = arr[K];
        j = K-1;

        while(temp <= arr[j]){
            arr[j+1] = temp;
            j--;
        }
    arr[j+1] = temp;
    K++;
    }
}

#endif
```



```
          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                    Insertion Sorting:
 21 38 41 49 52 57 84 108 132 134 136 153 156 173 188 212 281 283 288 292 304 325 328 334 334 340 353 363 365 383 387 394 394 417 423
 438 441 452 471 473 485 488 489 491 526 542 549 551 556 557 559 572 618 635 648 650 668 670 671 690 692 694 710 718 731 731 734 735
 737 739 747 750 774 778 785 789 828 831 834 843 843 851 870 871 874 903 905 905 914 931 944 950 951 953 960 970 970 973 988 997
```

OBSERVATION ANALYSIS

- Insertion Sort works by taking one element at a time and placing it in its correct position. I think among the three, this is the most efficient sorting especially when the order is already identified. Because it works by comparing each element with the elements before it and only shifting the elements that are greater, it does not need to repeatedly compare the entire list. This makes it efficient when the data is already sorted or nearly sorted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code

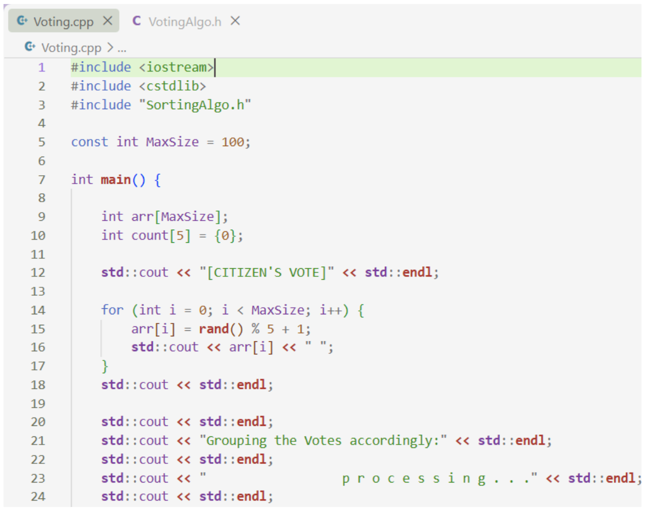
- Output Testing

PSEUDOCODE:

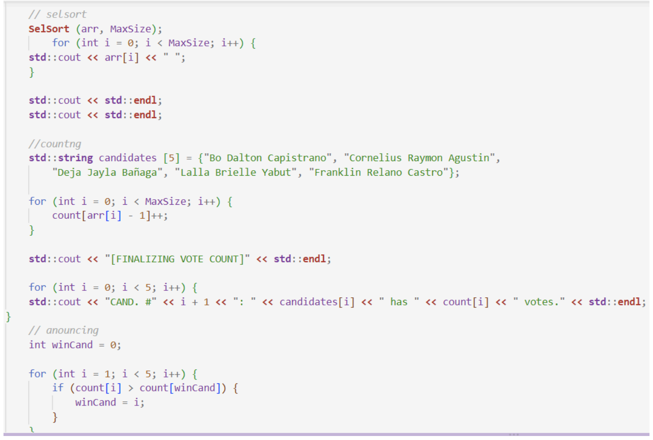
```
START PROGRAM

    DECLARE the array list as ARR with a maximum size of 100.
    DECLARE the array COUNT with 5 elements.

    DISPLAY “[CITIZEN’S VOTE]”

    FOR i = 0 to maximum size, DO
        ARR [i] = random number from 1 to 5
        DISPLAY arr[i]
    END FOR

    DISPLAY "Grouping the Votes Accordingly:"
    DISPLAY "p r o c e s s i n g . . . "

    CALL SelSort (arr, MaxSize)

    FOR i = 0 to 99, DO
        DISPLAY arr[i]
    END FOR

    DECLARE CANDIDATES as an array with 5 elements

    SET CANDIDATES[0] to "Bo Dalton Capistrano"
    SET CANDIDATES[1] to "Cornelius Raymon Agustin"
    SET CANDIDATES[2] to "Deja Jayla Bañaga"
    SET CANDIDATES[3] to "Lalla Brielle Yabut"
    SET CANDIDATES[4] to "Franklin Relano Castro"

    FOR i = 0 to 99, DO
        COUNT [arr[i] - 1], THEN increment
    END FOR

    DISPLAY "[FINALIZING VOTE COUNT]"

    FOR i = 0 to 4, DO,
        DISPLAY “CAND. #”, i + 1, “ :”, candidates and their voting counts.
    END FOR

     DECLARE winCand as an integer
     SET winCand to 0
```

```
        FOR i = 1 to 4, then DO
            IF count[i] > count[winCand], THEN
                SET winCand = I
            END IF
        END FOR

    DISPLAY "Congratulations! ", candidates[winCand], ", you're the winner!"
    DISPLAY "Total Votes accumulated: ", count[winCand], " votes."

END PROGRAM
```

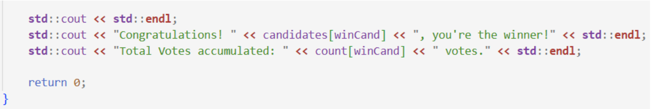
CODE:



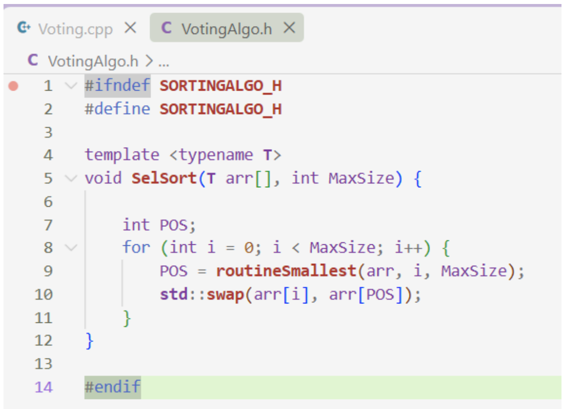
```
#include <iostream>
#include <cstdlib>
#include "SortingAlgo.h"

const int MaxSize = 100;

int main() {

    int arr[MaxSize];
    int count[5] = {0};

    std::cout << "[CITIZEN'S VOTE]" << std::endl;

    for (int i = 0; i < MaxSize; i++) {
        arr[i] = rand() % 5 + 1;
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    std::cout << std::endl;
    std::cout << "Grouping the Votes accordingly:" << std::endl;
    std::cout << std::endl;
    std::cout << "                   p r o c e s s i n g . . ." << std::endl;
    std::cout << std::endl;
```



```
    // selsort
    SelSort (arr, MaxSize);
        for (int i = 0; i < MaxSize; i++) {
    std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
    std::cout << std::endl;

    //countng
    std::string candidates [5] = {"Bo Dalton Capistrano", "Cornelius Raymon Agustin",
        "Deja Jayla Bañaga", "Lalla Brielle Yabut", "Franklin Relano Castro"};

    for (int i = 0; i < MaxSize; i++) {
        count[arr[i] - 1]++;
    }

    std::cout << "[FINALIZING VOTE COUNT]" << std::endl;

    for (int i = 0; i < 5; i++) {
    std::cout << "CAND. #" << i + 1 << ": " << candidates[i] << " has " << count[i] << " votes." << std::endl;
}
    // anouncing
    int winCand = 0;

    for (int i = 1; i < 5; i++) {
        if (count[i] > count[winCand]) {
            winCand = i;
        }
```

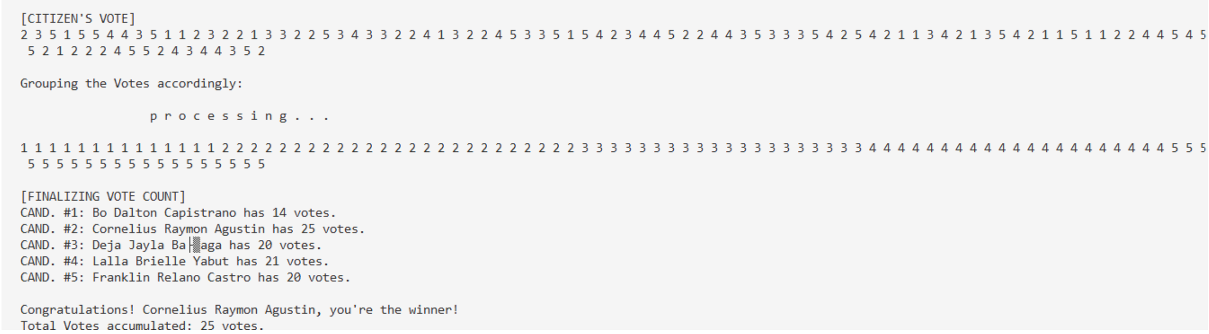


```
    std::cout << std::endl;
    std::cout << "Congratulations! " << candidates[winCand] << ", you're the winner!" << std::endl;
    std::cout << "Total Votes accumulated: " << count[winCand] << " votes." << std::endl;

    return 0;
}
```



```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

template <typename T>
void SelSort(T arr[], int MaxSize) {

    int POS;
    for (int i = 0; i < MaxSize; i++) {
        POS = routineSmallest(arr, i, MaxSize);
        std::swap(arr[i], arr[POS]);
    }
}

#endif
```

OUTPUT:



```
[CITIZEN'S VOTE]
2 3 5 1 5 5 4 4 3 5 1 1 2 3 2 2 1 3 3 2 2 5 3 4 3 3 2 2 4 1 3 2 2 4 5 3 3 5 1 5 4 2 3 4 4 5 2 2 4 4 3 5 3 3 3 5 4 2 5 4 2 1 1 3 4 2 1 3 5 4 2 1 1 5 1 1 2 2 4 4 5 4 5
 5 2 1 2 2 2 4 5 5 2 4 3 4 4 3 5 2

Grouping the Votes accordingly:

                  p r o c e s s i n g . . .

1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5
 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

[FINALIZING VOTE COUNT]
CAND. #1: Bo Dalton Capistrano has 14 votes.
CAND. #2: Cornelius Raymon Agustin has 25 votes.
CAND. #3: Deja Jayla Ba├▒aga has 20 votes.
CAND. #4: Lalla Brielle Yabut has 21 votes.
CAND. #5: Franklin Relano Castro has 20 votes.

Congratulations! Cornelius Raymon Agustin, you're the winner!
Total Votes accumulated: 25 votes.
```

CONCLUSION:

- **Learning new sorting algorithms threw me back at first because they were difficult for me to understand. However, to learn and master these sorting methods is very essential since the help and develop on making efficient methods while working on large amounts of data in computer programming and real-world applications. Just like in the supplementary activity, sorting and identifying the winner for the election is useful as it makes the organizing easier, comparing the results in a short amount of time and determining the winner. Although I think my method of time complexity still needs to be sharpened and developed—at least after analysing my work.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: ______________________________

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.